

School of Computing: assessment brief

Module title	Computer Graphics
Module code	COMP3811
Assignment title	Coursework 1
Assignment type and description	Programming assignment: Graphics fundamentals
Rationale	The coursework revolves around fundamental graphics operations and data types. These include images and the manipulation thereof, drawing primitives such as lines and triangles, and blitting images.
Page limit and guidance	Report: 8 A4 pages with 2cm or larger margins, 10pt font size (including figures). You are allowed to use a double-column layout. Code: no limit. Please read the submission instructions carefully!
Weighting	50%
Submission deadline	2024-11-07 14:00
Submission method	Gradescope: code and report
Feedback provision	Written notes
Learning outcomes assessed	Understanding, description and utilization of standard methods to programmatically create and manipulate images. Understanding, description, application and evaluation of fundamental algorithms and methods in computer graphics.
Module lead	Markus Billeter

1. Assignment guidance

In the first coursework, you are tasked with implementing several drawing functions for primitive graphics operations. These include drawing lines, triangles and blitting images. You will verify that these functions work correctly and analyze their behaviour.

Before starting your work, please study the coursework document in its entirety. Pay special attention to the requirements and submission information. Plan your work. It might be better to focus on a subset of tasks and commit to these fully than to attempt everything in a superficial way.

2. Assessment tasks

Please see detailed instructions in the document following the standardized assessment brief (pages i-iv). The work is split into several tasks, accounting for 50% of the total module grade.

3. General guidance and study support

Please refer to materials handed out during the module, specifically the tutorial-style exercises for 2D graphics and maths.

Support is primarily provided during scheduled lab hours. Support for general issues may be provided through the module's "Teams" channel. Do not expect answers outside of scheduled hours. Do not post specific issues relating to your code in the public Teams channels. Do not crosspost across multiple channels.

4. Assessment criteria and marking process

Submissions take place through Gradescope. Valid submissions will be marked primarily based on the report and secondarily based on the submitted code. See following sections for details on submission requirements and on requirements on the report. Marks and feedback will be provided through Minerva (and not through Gradescope - Gradescope is only used for submissions!).

5. Submission requirements

Your coursework will be graded once you have

- (a) Submitted required files (code and report) through Gradescope.
- (b) If deemed necessary, participated in an extended interview with the instructor(s) where you explain your submission in detail.

Your submission will consist of source code and a report ($\lesssim 8$ pages). *The report is the basis for assessment. The source code is supporting evidence for claims made in the report. Tasks/results without supporting code will receive zero marks.*

Submissions are made through Gradescope (do *not* send your solutions by email!). You can use any of Gradescope's mechanisms for uploading the complete solution and report. In particular, Gradescope accepts .zip archives (you should see the

contents of them when uploading to Gradescope). Do not use other archive formats (Gradescope must be able to unpack them!). Gradescope will run preliminary checks on your submission and indicate whether it is considered a valid submission.

The source code must compile and run as submitted on the standard SoC machines found in the UG teaching lab (2.05 in Bragg). Your code must compile cleanly, i.e., it should not produce any warnings. If there are singular warnings that you cannot resolve or believe are in error, you must list these in your report and provide an explanation of what the warning means and why it is acceptable in your case. This is not applicable for easily fixed problems and other bulk warnings (for example, type conversions) – you are always expected to correct the underlying issues for such. *Do not change the warning level defined in the handed-out code. Disabling individual warnings through various means will still require documenting them in the report.*

Your submission must not include any “extra” files that are not required to build or run your submission (aside from the report). In particular, you must *not* include build artifacts (e.g. final binaries, `.o` files, ...), temporary files generated by your IDE or other tools (e.g. `.vs` directory and contents) or files used by version control (e.g. `.git` directory and related files). Note that some of these files may be hidden by default, but they are almost always visible when inspecting the archive with various tools. Do not submit unused code (e.g. created for testing). Submitting unnecessary files may result in a deduction of marks.

*While you are encouraged to use version control software/source code management software (such as git or subversion), you must **not** make your solutions publicly available. In particular, if you wish to use Github, you must use a **private repository**. You should be the only user with access to that repository.*

6. Presentation and referencing

Your report must be a single PDF file called **report.pdf**. In the report, you must list all tasks that you have attempted and describe your solutions for each task. *Include screenshots for each task unless otherwise noted in the task description!* You may refer to your code in the descriptions, but descriptions that just say “see source code” are not sufficient. Do **not** reproduce bulk code in your report. If you wish to highlight a particularly clever method, a short snippet of code is acceptable. Never show screenshots/images of code - if you wish to include code, make sure it is rendered as text in the PDF using appropriate formatting and layout. Screenshots must be of good quality (keep the resolution at 1280×720 or higher, but scale them down in the PDF). Don’t compress the screenshots overly much (e.g., visible compression artifacts).

Apply good report writing practices. Structure your report appropriately. Use whole English sentences. Use appropriate grammar, punctuation and spelling. Provide figure captions to figures/screenshots, explaining what the figure/screenshot is showing and what the reader should pay attention to. Refer to figures from your main text. Cite external references appropriately.

Furthermore, the UoL standard practices apply:

The quality of written English will be assessed in this work. As a minimum, you must ensure:

- Paragraphs are used
- There are links between and within paragraphs although these may be ineffective at times
- There are (at least) attempts at referencing
- Word choice and grammar do not seriously undermine the meaning and comprehensibility of the argument
- Word choice and grammar are generally appropriate to an academic text

These are pass/ fail criteria. So irrespective of marks awarded elsewhere, if you do not meet these criteria you will fail overall.

7. Academic misconduct and plagiarism

You are encouraged to research solutions and use third-party resources. If you find such, you must provide a reference to them in your report (include information about the source and original author(s)). Never “copy-paste” code from elsewhere – all code must be written yourself. If the solution is based on third party code, make sure to indicate this in comments surrounding your implementation in your code, in addition to including a reference in your report. *It is expected that you fully understand all code that you hand in as part of your submission. You may be asked to explain any such code as part of the marking process. If deemed necessary, you may be asked to attend a short individual interview with the instructor(s), where you are asked to explain specific parts of your submission.*

Furthermore, the UoL standard practices apply:

Academic integrity means engaging in good academic practice. This involves essential academic skills, such as keeping track of where you find ideas and information and referencing these accurately in your work.

By submitting this assignment you are confirming that the work is a true expression of your own work and ideas and that you have given credit to others where their work has contributed to yours.

8. Assessment/marking criteria grid

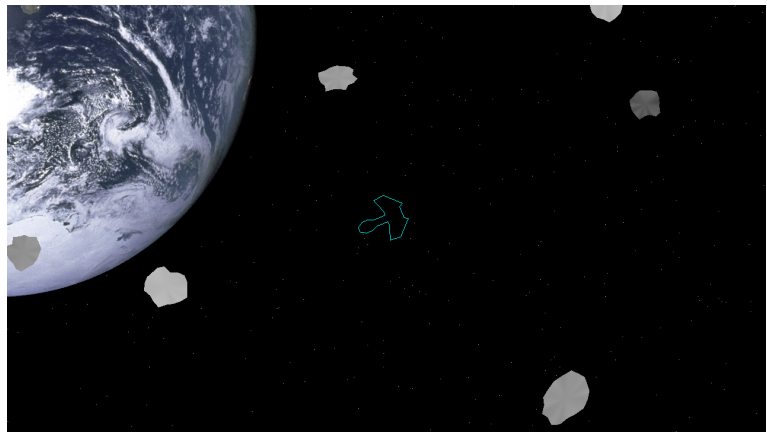
(See separate document.)

COMP3811

Coursework 1

Contents

1 Tasks	1
1.1 Setting Pixels	2
1.2 Drawing Lines	2
1.3 2D Rotation	3
1.4 Drawing triangles	4
1.5 Blitting images	4
1.6 Testing: lines	4
1.7 Testing: triangles	5
1.8 Benchmark: Blitting	5
1.9 Benchmark: Line drawing	6
1.10 Your own space ship	6



Coursework 1 focuses on basic graphics operations in 2D, including manipulating images, drawing lines and triangles, and [blitting](#) images. Coursework 1 is to be solved individually and determines 50% of the total mark for COMP3811.

Before starting work on the tasks below, study this document in its entirety. Plan your work. It is likely better to focus on a subset of tasks and commit to these fully than to attempt everything in a superficial way. For the purpose of planning, you may consider tasks in Sections 1.6 to 1.9 to be (more) advanced tasks. You may want to hold off on these until you have completed other tasks.

*While you are encouraged to use version control software/source code management software (such as git or subversion), you must **not** make your solutions publicly available. In particular, if you wish to use Github, you must use a private repository. You should be the only user with access to that repository.*

You are allowed to discuss ideas with your colleagues. However, do not share your code with anybody else. You must program independently and not base your submission on any code other than what has been provided with the coursework and/or in the exercises for COMP3811. As a special exception, you may reuse code from COMP3811 exercises that was handed out (including briefs) or that *you are the sole author of*.

Use good commenting practices to explain your approach and solution. Good, thoughtful and well-written comments will help you show that you understand your code. It will also decrease the chances of accidentally ending up with submissions similar to other's work.

Coursework 1 will not require you to use any third-party software outside of what is included in the handed-out code. You are therefore not allowed to use additional third-party libraries.

1 Tasks

Start by downloading the Coursework 1 base code. Make sure you are able to build it. If necessary, refer to the first exercise handed out in COMP3811. It uses the same base structure and includes detailed instructions to get you started.

The Coursework 1 base code includes several subprojects. Some of them you will have already encountered in exercises. Others are specific to the coursework.

- `main`, a program which combines elements from all tasks to draw a game-like 2D environment.
- `draw2d`, a library where you implement the various drawing functions.
- `vmllib`, a library for linear algebra/math-related functions.
- `support`, a library with some helper functions relating to setup and on-screen display.
- `lines-sandbox`, a simple graphical program that only draws lines. Use it for quick visual testing.
- `lines-test`, a program for automated tests relating to line drawing.
- `lines-benchmark`, a program for automated benchmarks relating to line drawing.
- `triangles-sandbox`, a graphical program that only draws triangles. Use it for quick visual testing.
- `triangles-test`, a program for automated tests relating to triangle drawing.
- `blit-benchmark`, a program for automated benchmarks relating to image blitting.
- `x-*`, which correspond to the various third-party libraries. Unlike the other subprojects, these are not defined in the main `premake5.lua`, but rather in `third_party/premake5.lua`.

Although the teaser image looks somewhat like a screenshot from a game, quite a few things that make a game are missing. This includes functionality like collision detection, sound, game logic, AI, networking, etc etc. However, most importantly for COMP3811, there are several graphics subroutines whose implementations are missing as well. Each task that you complete will progress you from the initial empty black screen towards the teaser image shown on the first page in this document.

Coursework 1 includes tasks for a maximum of 50 marks. Each of the tasks below indicates the maximum number of marks that you can receive for it. Grading of each task is assessed based on the descriptions and analysis in your report and further assessed based on: code quality, including correctness, clarity, commenting and efficiency. Your code must work in both debug and release modes.

1.1 Setting Pixels

2 marks

Drawing anything on screen ultimately requires you to set a specific pixel to a specific color. In this first task, you will implement helper functions to do so. Any drawing from here on out will use these helpers, specifically the `Surface::set_pixel_srgb` method.

Consider the `Surface::set_pixel_srgb` and `Surface::get_linear_index` methods. These are declared in the `Surface` class in `draw2d/surface.hpp` and defined in `draw2d/surface.inl`. Implement the two functions in `draw2d/surface.inl`.

The pixel coordinate $(0, 0)$ **must** correspond to the *bottom-left* corner in the window.

The `Surface` class uses a RGBx image format, where each color component is stored in a single 8-bit unsigned integer (`std::uint8_t`). You may set the fourth component ("x") to zero. It is included to pad each pixel to be 32-bits but otherwise ignored. The image is stored in row-major order.

Important:

- You are *not* allowed to change the `draw2d/surface.hpp` header (and, consequently, you may not change the interface of the `Surface` class).
- You must keep the `assert()`-line as the first line of the `Surface::set_pixel_srgb` function definition. Only add new code below it.

These methods are used to draw the background particle field. Refer to Figure 1 for possible results. You can move around by first tapping `space` to enter piloting mode (your mouse cursor should turn into a crosshair), moving the mouse cursor in the direction you wish to accelerate, and then pressing and holding the right mouse button to accelerate. Tapping space bar a second time will exit the piloting mode.

In your report: Include a screenshot of the particle field. Make sure that the particles are visible (if necessary, add a scaled-up cut-out image).

1.2 Drawing Lines

8 marks

Next, consider the function `draw_line_solid`. The function is declared in the `draw2d/draw.hpp` header and defined in the `draw2d/draw.cpp` source file. The function is supposed to draw a solid single-color line between the points `aBegin` and `aEnd`. The color of the line is specified by the function's final argument.

Implement the `draw_line_solid` function. The goal is to produce a line that is as thin as possible (single pixel width) and that does not have any holes (i.e., each pixel should connect to another pixel either by nearest neighbours or by diagonals). Recall the parametrised version of a line as a starting point. You should ensure

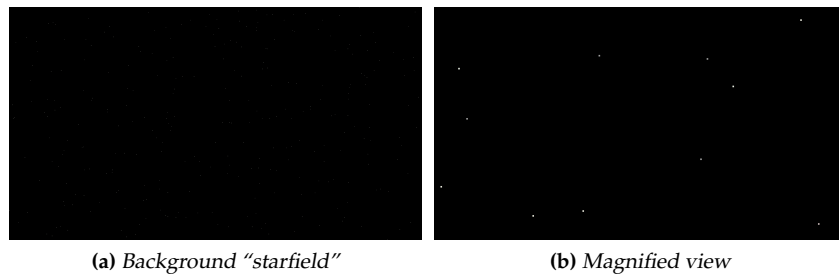


Figure 1: Task 1. You might need to zoom in to the left image in your PDF viewer to see the individual points. The right image shows a magnified view of the top-left region.

that the function produces correct results with all inputs, including cases where the line extends off-screen (so, you must include clipping). You may pick any drawing method, but it should scale with $O(N)$ with respect to the number of drawn pixels (N). You should not make any dynamic allocations (nor any system calls) in the line drawing method.

The handed-out code contains two additional programs related to your line drawing. Use `lines-sandbox` to visually verify your results in isolation. It includes a small number of examples already. You can switch between the examples using the number keys. See source code comments for more information. You are free to add additional examples.

The second program, `lines-test`, runs a few automated tests on the line drawing. It uses the [Catch2](#) testing library. Ensure that your implementation passes the existing tests. Refer to the source code for more information on the tests.

Note: You must not change the prototype of the `draw_line_solid` function, and you must use `Surface::▽ > set_pixel_srgb` to draw pixels.

With the line drawing in place, you should now be able to see a space ship (Figure 2a).

In your report: Explain your line drawing method (as a reminder: do not just dump code into your report). Document your handling of lines extending off-screen. Include a screenshot of the drawn ship.

1.3 2D Rotation

2 marks

The space ship initially always faces to the right. To make it turn, you must implement a few functions related to the 2×2 matrices:

- Matrix-matrix multiplication: `Mat22f operator*( Mat22f const&, Mat22f const& ) noexcept`
- Matrix-vector multiplication: `Vec2f operator*( Mat22f const&, Vec2f const& ) noexcept`
- Creation of a rotation matrix: `Mat22f make_rotation_2d( float aAngleInRadians ) noexcept`

The functions are both declared and defined in `vmllib/mat22.hpp`. Provide implementations for these functions/operators. With the implementations in place, the ship should now always face the mouse cursor when in piloting mode (compare to Figure 2b – the spaceship is facing to the bottom right in this example).

In your report: Include a screenshot of the rotated ship.

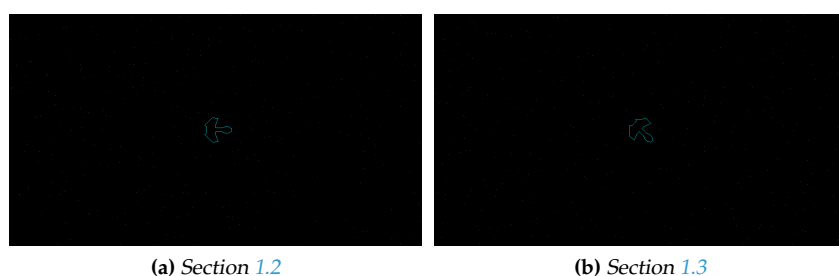


Figure 2: (a) Space ship without rotation, facing the default direction (right). (b) Space ship with rotation, always facing the mouse cursor when in piloting mode.

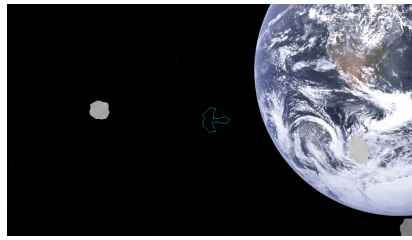


Figure 3: Approaching the earth (*lithobraking* not yet implemented!).

1.4 Drawing triangles

8 marks

Consider the function `draw_triangle_interp`. It is also declared in the `draw2d/draw.hpp` header and defined in `draw2d/draw.cpp`. This function draws a single triangle defined by its three vertices (`aP0`, `aP1` and `aP2`). Each vertex is assigned a color (`aC0`, `aC1` and `aC2`, respectively). These colors should be interpolated across the triangle with barycentric interpolation. Implement this function. Make sure that the function works correctly with all (reasonable) inputs.

Unlike earlier examples, the colors are specified in linear RGB (`ColorF`). You should perform the interpolation in linear RGB space and only convert to the 8-bit sRGB representation when writing the color value to the surface.

You can pick any method, but it should be reasonably efficient (e.g., simply testing all pixels in the screen is not sufficient). You should not make any dynamic allocations or system calls in the triangle drawing method.

Use the `triangles-sandbox` to visually experiment with your triangle drawing in isolation. Run the tests in `triangles-test` and ensure that they pass. When you have implemented the triangle drawing, you should also be able to see the asteroids in the main program (see teaser image).

Note: You must not change the prototype of the `draw_line_solid` function, and you must use `Surface::▽` `> set_pixel_srgb` to draw pixels.

In your report: Explain your method. Document any special handling that you perform. Include a screenshot of the main program, with the asteroids visible.

1.5 Blitting images

4 marks

In this task, you will implement image blitting with alpha masking. Consider the `blit_masked` function declared in `draw2d/image.hpp` and defined in `draw2d/image.cpp`. You will also need to implement a few helper functions in `draw2d/image.inl`. Search for lines containing the string `// TODO`.

You should blit the input image (a `Image` of type `ImageRGBA`) to the position specified by `aPosition`. The position is *relative to the center* of the input image. Input pixels with an alpha value (a component of the `Color_sRGB_Alpha` color struct) less than 128 should be discarded. Consider efficiency in your implementation and do not make any dynamic allocations/syscalls (etc etc.).

If you have implemented the method correctly, you should find the earth after flying a bit to the right – it will be off-screen initially (see teaser image and Figure 3).

Note: You must not change the prototype of the `blit_masked` function. You must not change the `ImageRGBA` class and the `load_image` function.

In your report: Describe your implementation of the blit. Discuss the efficiency of your implementation. Focus specifically on choices in your implementation that benefit efficiency and the impact of clipping/culling.

1.6 Testing: lines

8 marks

Consider the `lines-test` program. It contains a few example tests that verify expected behaviour. However, the tests are far from exhaustive.

Implement tests for the following four scenarios:

1. Consider a line from p_0 to p_1 . It should be identical to the line from p_1 to p_0 .

2. Consider lines with one point inside the surface and one outside.
3. Consider lines with both points outside of the surface.
4. Consider two lines. The first starts at p_0 and extends to p_1 . The second starts at p_1 and extends to p_2 . When both are drawn, there should be no gap between the two lines. Extend this to multiple lines - what happens if the lines are very short?

Each scenario must be implemented in a separate `TEST_CASE` in the corresponding source file (e.g., Scenario 1 is in `scenario-1.cpp` and so on). Each scenario is expected to test multiple different representative lines, for which you are required to make informed choices. It is likely you will need multiple assertions per test.

If your line drawing implementation fails some of the tests, you should tag the corresponding `TEST_CASE` with `⚡[!mayfail]`. Mention this in your report.

In your report: Document which tests you have added. Describe how you have implemented the test (what do you actually test?). List what representative lines you have chosen to include in your tests and motivate the choice of these. Where possible, sketches and/or screenshots (e.g. from `lines-sandbox`) that show your representative lines.

1.7 Testing: triangles

4 marks

Add at least two (2) more distinct test cases to the `triangles-test` program. Refer to Section 1.6 for details – the same requirements/guidelines apply here. Use the provided `scenario-N.cpp` files. Make sure the tests that you add are meaningful.

In your report: Document the tests that you have added. Explain the purpose of each test and why you included it. No marks will be awarded for tests that lack an explanation and solid reasoning. If possible, visualize the test case using sketches and/or screenshots (e.g., from `triangles-sandbox`).

1.8 Benchmark: Blitting

6 marks

Compare the performance of your blit (`blit_masked`) to two more blit variants under different conditions. For this task, use `blit-benchmark` which in turn uses [Google's microbenchmarking library](#) to allow you to implement these benchmarks. Study the documentation and examples at the provided link. The provided code implements a simple example that measures the performance for a simple blit operation.

Important: You should only ever benchmark code built in the *release* configuration. The *debug* configuration disables many compiler optimizations (including code inlining!) to aid debugging and is therefore not representative of the final performance. Hence, when running benchmarks, make sure you only ever use release builds.

Modern CPUs and operating systems also adjust clock rates of the processor based on work load. Many CPUs can additionally boost to higher clock rates for short periods of time. These features are obviously desirable under normal conditions, but make life during benchmarking more difficult. Refer to [Benchmarking details](#) below for additional discussion on this topic.



You should first implement the additional blit variants in `draw-ex.cpp`:

- `blit_ex_solid`: A blit without alpha masking, where you just copy over the target image pixel by pixel. Implement this yourself using loops in C++.
- `blit_ex_memcpy`: A blit without alpha masking, but implement this using `std::memcpy`, one for each line in the image.

These “extended” functions take a `SurfaceEx` argument instead of the `Surface`. The main difference is that `SurfaceEx` gives out a raw pointer to the image data; you will need this for (minimally) the `std::memcpy` `>`-based variant. Study the declaration of `SurfaceEx` for details.

Before performing any benchmarks, you should ensure that the variants work correctly. There is no point in benchmarking incorrect code.

Benchmark the performance under different conditions. Perform comparisons with a smaller (e.g., 128×128) and a larger (e.g., 1024×1024) input image. Perform comparisons on a smaller framebuffer (320×240), the default size (1280×720), full HD (1920×1080) and an 8k framebuffer (7680×4320). Vary only one variable at a time. (However, the benchmark program should run all variants automatically.)

Analyze your results. Do they seem realistic/reasonable? What are your observations? Can you explain what you see?

In your report: Mention what CPU you are running on. If you know, include information about your system RAM (amount and speed) and CPU caches. Present your results using graphs/plots (do not dump output from the terminal or -worse- screenshots of the terminal output in the report!). What are your observations? Try to explain what you have seen.

Marks are mainly awarded for a solid analysis and discussion of the results. No marks are awarded for just showing the results. Do not forget to include units on axes/reported numbers.

1.9 Benchmark: Line drawing

7 marks

Use the `lines-benchmark` program. Refer to Section 1.8 for details on the benchmarking application.

For this task, implement a second line drawing algorithm in `draw_ex_line_solid()` (in `draw-ex.cpp`). You can choose from the following options for this:

- Research an optimized line drawing method. It should be based on existing (technical) literature that you can reference.
- If you previously implemented a method like DDA (with floating point), aim for an integer-only method (e.g. Bresenham). If you already implemented an integer-only method, then implement a standard DDA with floats¹.
- Come up with a potential optimization yourself? It must be non-trivial. If you opt for this, your choice must be approved by the module leader - discuss your choice with the module leader during one of the labs. You will be required to provide a solid theoretical reason why this optimization would improve performance. This should also be included in the report.

Your improved method may use the `SurfaceEx` class.

Next, identify a few (3-4) different representative lines to benchmark. You are fairly free in your choices here, but you are expected to motivate your choices later. The cases should differ from each other conceptually. Also perform the tests with different framebuffer sizes (see Section 1.8). Again, vary only one variable at a time.

Use the tests to verify that your line drawing performs like $O(N)$ with respect to visible pixels.

In your report: Mention which line drawing algorithms you compare and highlight their differences. Document your representative lines and explain why you have picked these cases. Highlight what you believe distinguishes them from each other. Present your results using graphs/plots (do not dump output from the terminal or -worse- screenshots of the terminal output in the report!) Evaluate and analyze these. Discuss them and try to explain what you have seen. What are your conclusions? Marks are mainly awarded for a solid analysis and discussion of the results.

If you haven't already (Section 1.8), mention what CPU you are running on. If you know, include information about your system RAM (amount and speed) and CPU caches.

Marks are mainly awarded for a solid analysis and discussion of the results. No marks are awarded for just showing the results. Do not forget to include units on axes/reported numbers.

1.10 Your own space ship

1 mark

The default space ship shape is defined in `main/spaceship.cpp`. It consists of a number of points that are connected by lines.

Define your own custom space ship (see instructions in the source code). You must not use more than 32 points. The ship shape must show some amount of complexity and creativity. In your report, indicate if you have created a custom design and include a screenshot of your custom ship.

Please indicate in the source code (see comments) whether you would allow us to use your ship shape in future iterations of the COMP3811 module (for example as non-player ships). Your choice here does *not* affect the marking of this task.

¹In real-time graphics, we typically avoid doubles. They cost twice as much storage, may be significantly slower to compute, and the extra precision is seldom needed. In fact, there are often better methods to improve precision than just reaching for a more expensive type.

In your report: Include a screenshot of your ship. Briefly explain your design and mention the number of lines/points you used.

Wrapping up

Please double-check the submission requirements and ensure that your submission conforms to these. In particular, pay attention to file types (archive format and report format) and ensure that you have not included any unnecessary files in the submission. Make sure that you have tested your code (compile and run) in both debug and release modes.

Acknowledgements

The document uses icons from <https://icons8.com>: , , , . The “free” license requires attribution in documents that use the icons. Note that each icon is a link to the original source thereof.

Benchmarking details

As mentioned previously in this document, CPU frequency scaling can make benchmarking results less reliable. There are some tricks that may help.

A common workaround is to “warm up the CPU” by running a computationally heavy task for a short while before starting measurements. The computational load will cause the OS to transition the CPU to a higher clock rate. The main measurements should only take place after this warm up.

Google Benchmark seems to run benchmarks in the order they are declared (at least when using a single file). You can try to add a dummy benchmark at the start, whose results you discard, before running the main benchmarks. You can also try reordering individual benchmarks. If the results stay the same or very close, it is a good sign.

A better way would be to disable the CPU frequency scaling temporarily. Unfortunately, this is not always possible (it requires superuser privilege, which you probably shouldn’t have on the SoC computers).

Nevertheless, if you have your own Linux machine, you can run the following (it requires the `cpupower` tool):

```
$ cpupower frequency-info
analyzing CPU 0:
  driver: acpi-cpufreq
  ...
  available cpufreq governors: ondemand userspace powersave performance schedutil
  current policy: frequency should be within 2.20 GHz and 3.70 GHz.
                    The governor "schedutil" may decide which speed to use
                    within this range.
  ...
```

This lists the current CPU frequency governor (here: `schedutil`) and available governors. We’re interested in the `performance` governor, which simply runs at the max clock rate the CPU supports. You can activate it with (this probably requires superuser access):

```
$ sudo cpupower frequency-set -g performance
Password: hunter2
Setting cpu: 0
...
Setting cpu: 11
Following CPUs are offline:
12-15
cpupower set operation was not performed on them
```

(The exact output will depend on your CPU. If you have them, don’t worry about the “offline” CPUs, these likely correspond to cores that were disabled at the factory.)

Run the tests with the “`performance`” governor.

After you complete the tests, you will want to switch back to your default governor (here: “`schedutil`”). Using the “`performance`” governor for an extended time will likely make your CPU waste power and may cause devices like laptops to run hot.